

Failure-Aware and Replayable Rollout Infrastructure for Long-Horizon Agent Reinforcement Learning

Anonymous Author(s)
Institution
email@domain.edu

Abstract

Large language model (LLM) agents are increasingly trained through reinforcement learning (RL) in interactive, multi-turn environments involving tools, memory, browsers, code sandboxes, databases, mobile devices, and other stateful external systems. However, current Agent RL infrastructure often treats runtime failures, tool errors, environment states, and trajectory reproducibility as secondary engineering concerns. In practice, many high-cost rollouts fail not because the policy lacks task knowledge, but because the runtime enters an infinite loop, a tool silently times out, a browser or container state drifts, memory/context degrades, a worker disconnects, or the failure cannot be replayed and attributed.

This proposal argues that Agent RL requires a dedicated rollout substrate that manages agent-environment interaction as a first-class systems problem. We propose a *failure-aware and replayable rollout infrastructure* for long-horizon Agent RL. The proposed system introduces structured failure semantics, event-sourced trajectory logging, replayable tool and environment provenance, session-level watchdogs, partial trajectory salvage, and training-aware rollout scheduling. The goal is to convert runtime failures from noisy exceptions into structured artifacts that can improve debugging, scheduling, data quality, and ultimately reinforcement learning.

1 Motivation

Recent Agent RL systems such as AgentRL [5] and ProRL Agent [4] suggest a clear trend: the bottleneck of agentic reinforcement learning is shifting from pure GPU-side optimization to scalable, reliable, and reproducible rollout production. Unlike conventional RLHF or single-turn preference optimization, agentic RL requires policies to interact with long-horizon, partially observable, stateful environments through tool calls and natural language observations. The rollout system is no longer a passive data loader; it determines which tasks are sampled, which environments are allocated, which failures are observed, which partial trajectories are retained, and which signals eventually reach the optimizer.

Community and empirical evidence increasingly indicates that failures in production agent systems are often runtime failures rather than purely model reasoning failures. AgentRM [2], an analysis motivated by more than 40,000 GitHub issues across agent frameworks, identifies scheduling failures and context degradation as major classes of practical agent-system failures. An empirical study of 998 bug reports from CrewAI and LangChain [3] finds that modern LLM agent framework bugs concentrate heavily around lifecycle stages involving self-action and tool execution. Other recent work studies where LLM agents fail and how targeted feedback can help agents recover [1].

These observations expose a gap in current Agent RL infrastructure. A 30-turn rollout that fails at turn 27 because of a malformed tool call, stale memory entry, browser timeout, or unrecoverable environment state is not merely a failed sample. It is an expensive systems event containing potentially useful information: what state the agent reached, what action caused failure, whether the failure was model-attributable, whether

the environment was contaminated, whether the session can be replayed, and whether the partial trajectory should be penalized, retried, salvaged, or discarded.

The central claim of this proposal is therefore:

Agent RL infrastructure should treat runtime failures, tool interactions, and environment states as first-class training objects, not incidental execution artifacts.

2 Research Vision

We propose to study Agent RL infrastructure as an operating-system-like layer for agent rollouts. Analogous to how an operating system manages processes, resources, isolation, failures, and logs, an Agent RL rollout substrate should manage sessions, tools, environments, memory/context budgets, checkpoints, permissions, and replayable traces.

The proposed research focuses on three design principles.

Failure-native execution. Failures should be represented explicitly using typed semantics rather than unstructured exception strings. The system should distinguish model-attributable failures, tool failures, environment failures, scheduler cancellations, context-limit failures, and nondeterministic replay mismatches.

Replayable trajectories. Agent rollouts should be logged as event streams with model, token, tool, reward, and environment provenance. Replay should support exact replay, model-substituted replay, and environment-substituted replay.

Training-aware rollout management. The rollout scheduler should optimize useful learning signal rather than only raw request throughput. It should account for failure rates, reward variance, policy lag, environment cost, session age, buffer pressure, and tool reliability.

3 Research Questions

This project is organized around the following research questions:

1. **Failure representation.** How should runtime failures in long-horizon agent rollouts be represented, classified, and attributed?
2. **Replayability.** Can event-sourced trajectory logging improve debugging, failure recovery, data validation, and reproducibility for Agent RL?
3. **Rollout utility.** Can failure-aware rollout management increase the rate of useful trajectories compared with conventional capacity-based scheduling?
4. **Training semantics.** How should structured failure signals be incorporated into RL training without introducing harmful bias or reward hacking?
5. **Environment lifecycle.** Can environment snapshotting, reuse policies, and contamination checks improve throughput while preserving trajectory validity?

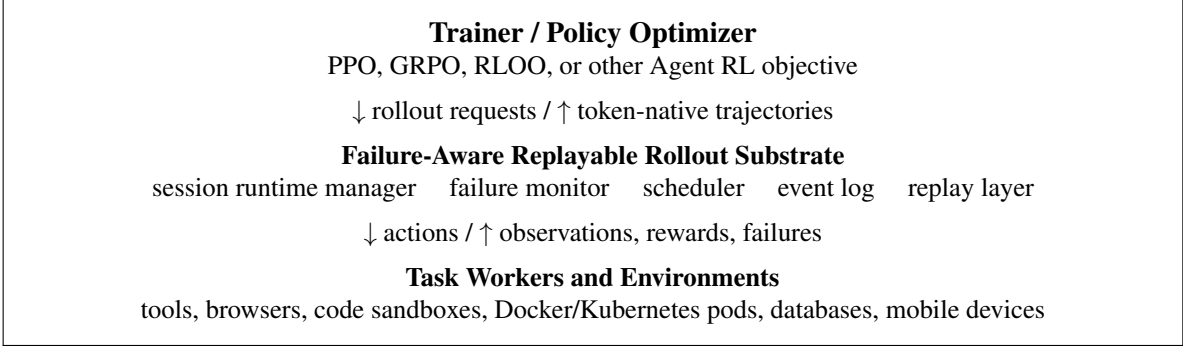


Figure 1: Conceptual architecture of the proposed rollout substrate.

4 Proposed System

Figure 1 summarizes the proposed system at a conceptual level. The system sits between RL trainers and task environments. It exposes a rollout API to trainers while internally managing sessions, workers, tools, environments, logs, failures, and replay.

4.1 Session Runtime Manager

The Session Runtime Manager treats each rollout as a managed runtime session. Each session has explicit metadata:

- session identifier, task identifier, and sample index;
- policy version, model checkpoint hash, tokenizer hash, and chat-template hash;
- allocated environment resources and tool permissions;
- current turn, token budget, wall-clock budget, and cost budget;
- failure state, cancellation state, and replay state.

The manager provides watchdog mechanisms for:

- per-turn and per-session timeouts;
- repeated action and no-progress loop detection;
- zombie session cleanup;
- rate-limit backoff;
- tool execution cancellation;
- partial trajectory salvage.

Unlike a simple timeout wrapper, the runtime manager does not merely kill failed sessions. It records why a session failed, whether the failure is attributable to the policy, whether the environment must be reset, and whether the partial trajectory can still be used for training or analysis.

Failure Type	Description	Primary Attribution
AGENT_INVALID_ACTION	Malformed JSON, invalid tool arguments, unsupported action, or schema violation.	policy
TOOL_TIMEOUT	Tool call exceeds execution budget.	tool/runtime
TOOL_EXECUTION_ERROR	Tool returns a typed runtime error.	tool/policy
ENV_CRASH	Browser, container, VM, pod, or external service becomes unavailable.	environment
ENV_CONTAMINATION	Environment state violates expected reset or isolation condition.	environment/runtime
CONTEXT_LIMIT	Session exceeds model context window or token budget.	policy/runtime
REPETITIVE_LOOP	Agent repeats semantically equivalent actions without progress.	policy/runtime
NO_PROGRESS	Observations or state metrics fail to improve across turns.	policy/environment
RATE_LIMIT	External API throttling or quota exhaustion.	infrastructure
SCHEDULER_CANCELLED	Session cancelled due to budget, preemption, or resource policy.	scheduler
UNKNOWN_RUNTIME_ERROR	Uncategorized failure.	unknown

Table 1: Initial failure taxonomy for long-horizon Agent RL rollouts.

4.2 Failure Taxonomy and Semantics

We propose a structured failure taxonomy for Agent RL rollouts. Table 1 shows an initial taxonomy.

Each failure type will have explicit operational semantics:

- whether the trajectory should be discarded, retained, retried, or salvaged;
- whether the failure should produce negative reward, neutral reward, or no training signal;
- whether the environment should be reset, snapshotted, or marked unhealthy;
- whether the policy, tool, environment, scheduler, or external API should be attributed;
- whether the failure is replayable.

This taxonomy is intended to be both a systems artifact and a training artifact. It allows the rollout infrastructure to support policy learning from failures while avoiding the mistake of penalizing the model for failures caused by infrastructure or environment nondeterminism.

4.3 Event-Sourced Trajectory Log

Current agent systems often store trajectories as flat message lists. This is insufficient for Agent RL because message lists omit important runtime information such as tool schemas, environment state, token spans, log probabilities, model versions, and failure provenance.

We propose to store each trajectory as an event stream. Representative event types include:

- SESSION_STARTED
- MODEL_REQUESTED
- MODEL_RESPONDED

- ACTION_PROPOSED
- TOOL_CALL_VALIDATED
- TOOL_CALL_EXECUTED
- OBSERVATION_RETURNED
- REWARD_EMITTED
- ENV_SNAPSHOT_CREATED
- FAILURE_DETECTED
- SESSION_CANCELLED
- SESSION_COMPLETED

Each event stores structured metadata:

- timestamp and parent event identifier;
- policy version, model checkpoint hash, and sampling parameters;
- tokenizer hash and chat-template hash;
- message span, token IDs, log probabilities, and loss mask;
- tool name, schema version, arguments, result, and error code;
- environment image digest, snapshot identifier, and health state;
- reward, metrics, and failure label where applicable.

This representation allows the same trajectory to serve multiple purposes: training data, debugging trace, replay script, failure dataset, and scheduler feedback.

4.4 Token-Native Rollout Records

Many agent frameworks are message-native: they store text messages and tokenize them later during training. This introduces risks such as chat-template drift, tool-call parsing drift, tokenizer mismatch, and loss-mask misalignment. We propose a token-native trajectory format in which rollout workers record the exact token IDs, log probabilities, and loss masks used by the policy at generation time.

The token-native record maintains both message-level and token-level views:

- message-level events support human inspection, tool replay, and task-level debugging;
- token-level tensors support policy-gradient training without reinterpreting the trajectory.

The infrastructure will include a drift validator that checks whether replay-time tokenization matches rollout-time tokenization. Any mismatch can be flagged before the trajectory reaches the trainer.

4.5 Replay Layer

The replay layer supports three modes:

Exact replay. The system replays the original trajectory using recorded model outputs, tool results, environment snapshots, and rewards. This mode is used for debugging and auditability.

Model-substituted replay. The system replays the same environment and observation stream with a different model checkpoint. This mode asks whether a newer policy would avoid the same failure.

Environment-substituted replay. The system replays the same agent actions against a fresh or restored environment. This mode detects environment contamination, nondeterminism, and tool instability.

Replay need not be perfectly deterministic for all environments. Instead, the system will explicitly classify replay determinism levels:

- exact deterministic replay;
- deterministic replay up to external API variation;
- state-equivalent replay;
- partial replay only;
- non-replayable trajectory.

4.6 Failure-Aware Rollout Scheduler

Existing rollout controllers commonly dispatch tasks by worker capacity or queue order. However, Agent RL rollouts differ in cost and learning value. Some tasks are slow but informative; some frequently timeout; some have high reward variance; some use expensive environments; some are produced by stale policies.

We propose a scheduler that uses training-aware signals:

- historical task success and failure rates;
- expected trajectory length;
- environment cost and availability;
- reward variance and task diversity;
- policy lag and rollout staleness;
- buffer pressure and trainer consumption rate;
- tool timeout rate and environment health.

Candidate scheduling policies include:

- FIFO scheduling;
- capacity-only scheduling;
- failure-rate-aware scheduling;
- cost-aware scheduling;
- reward-variance-aware scheduling;
- adaptive policy-lag scheduling.

The central metric is not raw rollout throughput, but useful training signal per unit cost.

5 Integration Plan

The proposed system can be implemented as an extension to an AgentRL-style architecture. AgentRL already separates the controller, task worker, environment controller, trainer, actor workers, reference workers, and rollout workers [5]. This makes it a suitable baseline substrate.

The proposed integration includes:

- extending task workers to emit typed failure events;
- adding event-sourced trajectory logging between controller and worker;
- adding session watchdogs and loop detectors to the controller;
- extending environment controllers with snapshot, health, and contamination metadata;
- adding a replay CLI for exact and substituted replay;
- adding a scheduler plugin interface for failure-aware dispatch;
- exporting token-native trajectories to PPO, GRPO, RLOO, or other RL trainers.

The design should remain trainer-agnostic. It should support AgentRL’s native trainer, verl-style trainers, or custom PPO/GRPO implementations.

6 Experimental Plan

6.1 Tasks

The evaluation should cover multiple failure modes:

- **Synthetic tool task.** A controlled environment for injecting invalid action, timeout, schema, and retry failures.
- **WebShop or AgentBench-style task.** A multi-turn tool-use task with delayed reward and realistic interaction traces.
- **Browser-based task.** A task involving browser state, repeated actions, page-load timeouts, and visual observations.
- **Code or shell sandbox task.** A task involving execution timeouts, permission errors, filesystem side effects, and partial progress.
- **Optional SWE-like long-horizon task.** A more expensive task for studying replay, partial trajectory salvage, and state consistency.

6.2 Baselines

We will compare against:

- standard capacity-based rollout dispatch;
- message-only trajectory logging;

- discard-on-failure trajectory handling;
- fixed timeout policy;
- no replay support;
- no environment snapshot or contamination check.

6.3 System Variants

The proposed system will be evaluated through ablations:

- runtime manager only;
- event-sourced logging only;
- replay layer only;
- failure-aware scheduler only;
- token-native validation only;
- full system.

7 Metrics

7.1 Systems Metrics

- completed trajectories per hour;
- useful trajectories per hour;
- failed trajectory ratio;
- recovered or salvaged trajectory ratio;
- zombie session rate;
- average environment utilization;
- GPU idle time due to rollout bottlenecks;
- P50/P95/P99 rollout latency;
- replay success rate;
- logging overhead.

7.2 Training Metrics

- reward curve;
- task success rate;
- sample efficiency;
- invalid action rate;
- tool error rate;
- context-limit failure rate;
- variance across random seeds.

7.3 Debugging and Reproducibility Metrics

- percentage of failures with structured attribution;
- time-to-root-cause in controlled debugging studies;
- replay determinism level;
- trajectory diff accuracy;
- percentage of trajectories passing token-drift validation.

7.4 Cost Metrics

- environment-hours per successful trajectory;
- tokens per successful trajectory;
- tool calls per successful trajectory;
- failed rollout cost;
- useful reward events per dollar-equivalent cost.

8 Expected Contributions

This project aims to make four primary contributions.

A failure taxonomy for long-horizon Agent RL rollouts. The taxonomy will classify runtime, tool, environment, context, scheduler, and replay failures, and specify their training and operational semantics.

A replayable trajectory substrate. The event-sourced logging layer will preserve model, token, tool, reward, and environment provenance, enabling replay, debugging, validation, and attribution.

A failure-aware rollout manager. The runtime manager will support timeout handling, loop detection, zombie cleanup, partial trajectory salvage, cancellation, and structured failure reporting.

A training-aware scheduler. The scheduler will optimize useful training signal rather than raw request throughput by accounting for failures, cost, reward variance, policy lag, and buffer pressure.

9 Why This Is Academically Interesting

The proposed work is not merely an engineering improvement to an agent framework. It addresses a foundational mismatch between traditional RL abstractions and modern LLM agent rollouts.

In standard RL, an environment step returns an observation, reward, and termination flag. In Agent RL, a step may involve a model-generated tool call, schema validation, browser state, memory retrieval, external API latency, invalid actions, execution side effects, reward delays, environment contamination, and partial failures. The rollout system therefore becomes part of the learning algorithm: it controls which trajectories are produced, which failures are observed, how failures are attributed, and which signals reach the optimizer.

This project reframes Agent RL infrastructure as a research object. It studies how runtime semantics, replayability, and scheduling affect training data quality and learning efficiency. The expected result is an infrastructure substrate that improves not only engineering reliability, but also the scientific reproducibility of Agent RL experiments.

10 Risks and Mitigations

Nondeterministic environments may be difficult to replay. We will support graded replay determinism rather than requiring perfect replay for all tasks. Partial replay and state-equivalent replay can still provide useful debugging and validation.

Failure-aware reward shaping may bias training. We will separate model-attributable failures from infrastructure-attributable failures and compare discard, neutral-label, and penalty policies through ablations.

Logging may reduce throughput. We will use asynchronous event logging, configurable trace levels, compact binary tensor storage, and sampling-based full tracing for high-volume runs.

The system may become too broad. The first paper will focus on failure taxonomy, replayable trajectories, and runtime management. Advanced scheduling and environment snapshotting can be staged as extensions if needed.

11 Timeline

12 Positioning

This project is complementary to AgentRL [5], ProRL Agent [4], AgentFly-style scalable agent frameworks, and trainer-oriented systems such as verl. It does not aim to be another monolithic Agent RL trainer. Instead, it targets the missing substrate beneath agentic training: reliable, failure-aware, replayable rollout production.

A concise positioning statement is:

We propose an operating-system layer for Agent RL rollouts, where sessions, tools, environments, failures, and trajectories are managed as structured runtime objects. Unlike prior work that focuses mainly on training throughput or agent orchestration APIs, our system improves the reliability, reproducibility, and learning value of long-horizon agent interactions.

Period	Milestones
Month 1	Reproduce AgentRL-style baseline; select tasks; implement initial event schema; collect baseline failure traces.
Month 2	Implement typed failure reporting, watchdogs, loop detection, timeout handling, and partial trajectory salvage.
Month 3	Implement exact replay, model-substituted replay, trajectory diff, and token-drift validation.
Month 4	Integrate with PPO/GRPO training; implement failure-aware scheduling policies; run controlled synthetic experiments.
Month 5	Run multi-task experiments; evaluate throughput, useful trajectory rate, training curves, and debugging metrics.
Month 6	Finalize ablations, write paper, package artifact, and prepare submission.

Table 2: Proposed six-month research timeline.

References

- [1] Anonymous. Where llm agents fail and how they can learn from failures, 2025. Author list to be completed from arXiv metadata.
- [2] Anonymous. Agentrm: An os-inspired resource manager for llm agent systems, 2026. Author list to be completed from arXiv metadata.
- [3] Anonymous. An empirical study of bugs in modern llm agent frameworks, 2026. Author list to be completed from arXiv metadata.
- [4] NVIDIA Research. Prorl agent: Rollout-as-a-service for rl training of multi-turn llm agents, 2026. Author list to be completed from camera-ready metadata.
- [5] Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Iat Long Iong, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, Rui Lu, Hongning Wang, Jie Tang, and Yuxiao Dong. Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework, 2025.